Workflow and architecture patterns for working together 2: Architectures

Jacob Archambault

May 23, 2026

Outline

Outline

Challenges

- multiple teams working in the same repository

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays
  - merge conflict regressions

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays
  - merge conflict regressions
  - sunk costs fallacy risks

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays
  - merge conflict regressions
  - sunk costs fallacy risks
- coupling of all services at the database layer

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays
  - merge conflict regressions
  - sunk costs fallacy risks
- coupling of all services at the database layer
  - high storage costs

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays
  - merge conflict regressions
  - sunk costs fallacy risks
- coupling of all services at the database layer
  - high storage costs
  - single point of failure risk

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays
  - merge conflict regressions
  - sunk costs fallacy risks
- coupling of all services at the database layer
  - high storage costs
  - single point of failure risk
  - risk of cross-project interference

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
  - feedback delays
  - merge conflict regressions
  - sunk costs fallacy risks
- coupling of all services at the database layer
  - high storage costs
  - single point of failure risk
  - risk of cross-project interference

Canonical solutions to these challenges

- multiple teams $\rightarrow$ inverse Conway maneuver

Canonical solutions to these challenges

- multiple teams $\rightarrow$ inverse Conway maneuver
- collaboration $\rightarrow$ business stream-aligned teams over activity oriented teams

Canonical solutions to these challenges

- multiple teams $\rightarrow$ inverse Conway maneuver
- collaboration $\rightarrow$ business stream-aligned teams over activity oriented teams
- late integration $\rightarrow$ adopting a true continuous integration workflow

Canonical solutions to these challenges

- multiple teams $\rightarrow$ inverse Conway maneuver
- collaboration $\rightarrow$ business stream-aligned teams over activity oriented teams
- late integration $\rightarrow$ adopting a true continuous integration workflow
- database coupling $\rightarrow$ Bezos API mandate/"Microservices"

Canonical solutions to these challenges

- multiple teams $\rightarrow$ inverse Conway maneuver
- collaboration $\rightarrow$ business stream-aligned teams over activity oriented teams
- late integration $\rightarrow$ adopting a true continuous integration workflow
- database coupling $\rightarrow$ Bezos API mandate/"Microservices"

Outline

- compare total commits across team branch vs. others - compare non-merge commits to non-PR merge commits across team branch vs. others (this represents percentage of value-adding work to maintenance
work) - compare time to merge across SpecFlow branches vs. others

Outline

- compare passing to failing pipeline runs across team branch vs.

others

Outline

Team composition

- all teams working in same git repository are in the same daily scrum

Team composition

- all teams working in same git repository are in the same daily scrum
- each team in the above sense has its own Microsoft Teams chat with only its own contributors in it

Team composition

- all teams working in same git repository are in the same daily scrum
- each team in the above sense has its own Microsoft Teams chat with only its own contributors in it
- these teams are *stream-aligned*, i.e., they focus on a particular business area (in our case, a transaction type)

Team composition

- all teams working in same git repository are in the same daily scrum
- each team in the above sense has its own Microsoft Teams chat with only its own contributors in it
- these teams are *stream-aligned*, i.e., they focus on a particular business area (in our case, a transaction type)
- these teams are *autonomous*, i.e., they can move work forward without requiring outside assistance from another team

Team composition

- all teams working in same git repository are in the same daily scrum
- each team in the above sense has its own Microsoft Teams chat with only its own contributors in it
- these teams are *stream-aligned*, i.e., they focus on a particular business area (in our case, a transaction type)
- these teams are *autonomous*, i.e., they can move work forward without requiring outside assistance from another team

Outline

- every git commit message begins with a Jira ticket number

- every git commit message begins with a Jira ticket number
- every commit is pushed immediately to origin

- every git commit message begins with a Jira ticket number
- every commit is pushed immediately to origin

Outline

Branching

- all commit work done by a given team, by both developers and testers, is committed directly to the same shared branch

Branching

- all commit work done by a given team, by both developers and testers, is committed directly to the same shared branch
-

Outline

- daily PRs - stacked PRs

Outline

- pipeline is definitive for deployment - daily stale branch job

Outline

Outline

Outline